

인터뷰 키트 · 주문·재고 API v2

제출 [@ dbff20e9379a5f61aca6317451221a539e53dfbf](https://github.com/wanted-hack-inchyangv/gaeun-order-api)

기준 버전 v2-be07fb44

워크벤치 <https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?tab=questions>

질문 14개 · 필수 4개 · 권장 5개 · 선택 5개 · 기본 질문 3개

LLM이 쓴 문장 중 3개가 질문 검사에 걸려 기본 질문으로 바꿨습니다.

이력서 연결 질문에는 지원자 이력서의 인용이 들어 있습니다. 복사한 내용은 면접 담당자에게만 공유해 주세요.

45분 진행안 구간 합 44분

도입 5분	
실패 디브리핑 16분	Q1 필수 · 실패 디브리핑 · 8분 Q2 필수 · 실패 디브리핑 · 8분
테스트 설계 6분	Q3 필수 · 테스트 설계 · 6분
설계 트레이드오프 6분	Q6 권장 · 설계 트레이드오프 · 6분
강점 확인 6분	Q4 필수 · 강점 확인 · 6분
지원자 질문 5분	

60분 진행안 구간 합 58분

도입 5분	
실패 디브리핑 24분	Q1 필수 · 실패 디브리핑 · 8분 Q2 필수 · 실패 디브리핑 · 8분 Q5 권장 · 실패 디브리핑 · 8분
테스트 설계 6분	Q3 필수 · 테스트 설계 · 6분
설계 트레이드오프 6분	Q6 권장 · 설계 트레이드오프 · 6분
강점 확인 12분	Q4 필수 · 강점 확인 · 6분 Q7 권장 · 강점 확인 · 6분
지원자 질문 5분	

Q1 필수 실패 디브리핑 원인 분석 · 8분

R-05 실행 기록에서 같은 키로 두 번 보냈을 때 두 번째 응답의 id가 첫 번째와 달랐고 재고도 두 번 줄었습니다. 주문 생성 요청이 들어왔을 때 먹등성 기록을 어떤 순서로 확인하고 갱신하도록 코드를 구성하셨나요?

의도 먹등 재전송에서 같은 키를 어떻게 조회·기록하는지, 그 순서가 중복 주문과 중복 차감을 어떻게 막는지 설명하는지 확인한다.

꼬리 질문

1. POST /orders 처리 흐름에서 키를 조회하는 시점과 기록을 남기는 시점은 각각 어디인가요?
2. 재고 차감과 먹등성 기록 저장이 서로 다른 시점에 일어날 때 어떤 불일치가 생길 수 있나요?
3. 기록을 남기는 시점을 앞당기거나 늦출 때 각각 어떤 비용이 생기나요?

좋은 답변의 신호

- 키 조회와 기록 저장의 순서를 코드 위치와 함께 구체적으로 설명한다.
- 두 번째 호출이 새 주문을 만들지 않으려면 무엇을 먼저 확인해야 하는지 스스로 짚는다.

우려 신호

- 재고 차감만 언급하고 먹등성 기록의 조회·저장 시점은 말하지 않는다.
- 응답 id가 달라진 이유를 코드 흐름과 연결하지 못한다.

근거

R-05 · 먹등 재전송

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-05>

R-05 재생

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-05&run=9dc6589f-002c-4a44-837d-ad8c00ab8253>

src/app.ts:23-31

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L23-L31>

src/app.ts:43-69

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L43-L69>

src/app.ts:33-41

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L33-L41>

R-06 실행 기록에서는 같은 키에 다른 본문을 보냈을 때 201과 409가 돌아왔고 재고도 함께 줄었습니다. 같은 키로 들어온 요청의 본문이 이전과 다를 때 어떤 값을 비교해 어떻게 판단하도록 작성하셨나요?

의도 키 충돌 판정에서 무엇을 비교 대상으로 삼았는지, 충돌 시 새 주문과 재고 변경을 어떻게 막는지 확인한다.

꼬리 질문

1. 저장해 둔 요청 내용과 새 요청을 비교할 때 어떤 필드를 기준으로 삼았나요?
2. 본문 비교를 어느 시점에 수행하면 재고 차감 전에 충돌을 가려낼 수 있나요?
3. 본문을 그대로 저장하는 방식과 해시로 저장하는 방식의 차이는 무엇인가요?

좋은 답변의 신호

- 충돌 판정에 쓰는 비교 대상과 그 판정 위치를 코드 흐름으로 설명한다.
- 충돌로 끝난 요청이 재고와 주문 상태를 바꾸지 않아야 하는 이유를 말한다.

우려 신호

- 충돌 판정 없이 재고 검사만 먼저 수행한 흐름을 그대로 답한다.
- 다른 본문이 왜 다른 오류 코드로 이어져야 하는지 설명하지 못한다.

근거

R-06 · 먹등 키 충돌

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-06>

R-06 재생

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-06&run=d9d1a193-07f6-4cdc-9cac-ef3a1d4fe5f6>

src/app.ts:43-69

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L43-L69>

src/app.ts:23-31

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L23-L31>

src/app.ts:33-41

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L33-L41>

제출한 테스트는 기존 상태에서 통과했지만 재고 부족 검사를 없앤 변형을 잡아내지 못했습니다. 재고 부족과 수량 경계를 검증하는 테스트를 어떤 입력과 기대값으로 설계하셨나요?

의도 경계·검증 규칙을 겨냥한 테스트 케이스를 어떤 입력과 단언으로 구성했는지, 변형을 잡아낼 만큼 촘촘한지 확인한다.

꼬리 질문

1. 재고와 같은 수량, 재고보다 큰 수량, 0과 음수 수량을 각각 어떤 케이스로 다루었나요?
2. 재고가 변하지 않았다는 사실을 어떤 조희로 단언하나요?
3. 검증 규칙이 사라졌을 때 실패할 단언을 지금 테스트에 추가한다면 무엇을 넣겠어요?

좋은 답변의 신호

- 경계값과 그 기대 상태 코드를 케이스별로 구분해 설명한다.
- 재고 불변을 확인하는 단언을 테스트에 포함해 말한다.

우려 신호

- 정상 주문 성공 케이스만 언급하고 거절 케이스는 말하지 않는다.
- 단언 없이 상태 코드만 확인하는 수준에 머문다.

근거

G1 · 테스트 실효성: 경계·검증

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=G1>

변이 M-01

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=G1&mutation=M-01>

src/app.ts:61-61

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L61-L61>

Q4 필수 강점 확인 경계 조건과 실패 처리 · 6분

R-03은 재고 부족과 재고 0, 재고와 같은 수량 주문까지 모두 기대대로 통과했습니다. 재고와 요청 수량을 비교하는 조건을 어떤 기준으로 정하셨나요?

의도 재고 부족 판정의 비교 기준과 경계 포함 여부를 어떤 근거로 정했는지 확인한다.

꼬리 질문

1. 재고와 같은 수량을 허용한 이유는 무엇인가요?
2. 거절로 끝난 요청에서 재고를 건드리지 않으려면 차감을 어디에 두어야 하나요?
3. 재고 확인과 차감 사이에 다른 요청이 끼어들면 어떤 문제가 생기나요?

좋은 답변의 신호

- 비교 연산의 경계 포함 여부를 명시적으로 설명한다.
- 거절 경로에서 재고 변경이 일어나지 않는 구조를 코드 흐름으로 말한다.

우려 신호

- 경계값을 우연히 맞춘 듯 설명하지 못한다.
- 거절과 차감의 순서를 구분하지 않는다.

근거

R-03 · 재고 부족 거절

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-03>

R-03 재생

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-03&run=e23d429c-8766-4c44-8f46-a9120ed1e3ae>

src/app.ts:43-69

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L43-L69>

src/app.ts:23-31

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L23-L31>

Q5 권장 실패 디브리핑 원인 분석 · 8분

R-09 실행 기록에서는 취소 후 재고가 시드 값으로 돌아오지 않고 1에 머물렀습니다. 취소 요청을 처리할 때 주문 상태와 재고를 어떤 순서로 갱신하도록 작성하셨나요?

의도 취소 처리에서 상태 변경과 재고 복구의 순서, 중복 취소 방지 조건을 어떻게 구성했는지 확인한다.

꼬리 질문

1. 취소 가능 여부를 판단하는 조건은 무엇인가요?
2. 이미 취소된 주문에 다시 취소가 들어오면 재고 복구가 두 번 일어나지 않게 어떻게 막나요?
3. 존재하지 않는 주문 id의 취소는 어느 지점에서 걸러지나요?

좋은 답변의 신호

- 상태 확인과 재고 복구의 순서를 코드 위치와 함께 설명한다.
- 중복 취소 시 복구가 한 번만 일어나야 하는 이유를 말한다.

우려 신호

- 재고 복구 로직이 실행되지 않은 이유를 코드에서 찾지 못한다.
- 중복 취소와 미존재 주문 처리를 같은 조건으로 뭉뚱그린다.

근거

R-09 · 주문 취소와 재고 복구

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-09>

R-09 재생

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-09&run=a07f6a1b-2daf-40eb-9007-897124dd13f8>

src/app.ts:43-69

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L43-L69>

src/app.ts:23-31

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L23-L31>

src/app.ts:71-87

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L71-L87>

src/app.ts:33-41

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L33-L41>

가장 긴 함수가 POST /orders 콜백으로 27줄이라는 신호가 있습니다. 이 콜백 안에서 요청 파싱, 검증, 재고 차감, 멍등성 처리를 어떤 기준으로 나누거나 합치셨나요?

의도 HTTP 처리와 도메인 규칙의 경계를 어떤 기준으로 나눴는지, 그 판단의 근거를 확인한다.

꼬리 질문

1. 이 콜백에서 HTTP와 무관한 규칙은 어디에 두었나요?
2. 규칙을 별도 함수로 빼면 어떤 점이 달라지나요?
3. 지금 구조에서 새 검증 규칙을 추가하면 어느 파일을 고치게 되나요?

좋은 답변의 신호

- 콜백이 담당하는 책임과 도메인 규칙의 경계를 구분해 설명한다.
- 분리했을 때의 이득과 비용을 함께 말한다.

우려 신호

- 모든 처리가 한 함수에 있는 이유를 설명하지 못한다.
- 도메인 규칙이 요청·응답 객체에 의존하는지 여부를 인식하지 못한다.

근거

src/app.ts:43-69

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L43-L69>

R-12 · 설계·변경 용이성

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-12>

R-08은 서로 다른 키 10건 동시 요청에서 201 5건과 409 5건, 재고 0으로 기대대로 통과했습니다. 동시에 들어온 요청들 사이에서 재고 확인과 차감이 어긋나지 않게 어떤 방식으로 처리하셨나요?

의도 동시 요청에서 재고 하한을 지키기 위해 어떤 직렬화나 원자적 처리를 택했는지 확인한다.

꼬리 질문

1. 재고 확인과 차감 사이에 다른 요청이 끼어들 수 있는 지점은 어디인가요?
2. 차감을 원자적으로 만들기 위해 어떤 언어 기능을 썼나요?
3. 요청 수가 늘어날 때 이 방식의 처리량은 어떻게 달라지나요?

좋은 답변의 신호

- 확인과 차감 사이의 경합 지점을 정확히 짚는다.
- 동시성 제어 방식과 그 한계를 함께 설명한다.

우려 신호

- 단일 스레드라 문제가 없다는 말로만 끝낸다.
- 재고가 음수가 될 수 있는 경로를 인식하지 못한다.

근거

R-08 · 다른 키 동시 요청과 재고 하한

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-08>

R-08 재생

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-08&run=52a00cd4-55d6-4c38-9f58-0cfcf29a5af1>

src/app.ts:43-69

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L43-L69>

src/app.ts:23-31

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L23-L31>

src/app.ts:33-41

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L33-L41>

같은 서비스를 여러 인스턴스로 띄워 요청이 서로 다른 인스턴스로 들어오는 상황을 가정해 보겠습니다. 지금 구현에서 가장 먼저 바뀌어야 할 부분은 어디인가요?

의도 요구사항이 바뀌었을 때 현재 구현(R-01·R-05·R-06)에서 영향을 받는 부분과 필요한 보장을 판단하는지 확인한다.

꼬리 질문

1. 지금 구현에서 그 상황의 영향을 받는 상태는 무엇인가요?
2. 그 상태를 안전하게 다루려면 어떤 보장이 필요한가요?
3. 그 보장을 얻는 방법들 가운데 어떤 기준으로 하나를 고르시겠어요?

좋은 답변의 신호

- 영향을 받는 상태와 코드 위치를 구체적으로 짚는다.
- 선택지마다 비용과 보장 수준을 비교해 하나를 고른다.

우려 신호

- 특정 기술 이름만 말하고 필요한 보장을 설명하지 않는다.
- 현재 구현이 받는 영향을 구체적으로 짚지 않는다.

근거

R-01 · 정상 주문 생성

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-01>

R-05 · 맥등 재전송

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-05>

R-06 · 맥등 키 충돌

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-06>

팀 프로젝트에서 멍등성 키를 도입해 같은 요청이 두 번 처리되지 않도록 설계하셨다고 하셨는데, 그때 키와 응답을 어디에 어떤 조건으로 저장하도록 정하셨나요?

이력서: 팀 프로젝트로 주문/예약 API를 설계하며 대용량 트래픽 환경을 고려한 동시성 제어와 멍등성 설계 경험을 쌓음

의도 이력서에 적힌 멍등성 설계 경험의 저장 위치·만료·충돌 처리 조건을 구체적으로 확인한다.

꼬리 질문

1. 같은 키로 다른 본문이 들어오면 어떤 응답을 주도록 정했나요?
2. 같은 키의 요청이 동시에 두 번 도착할 때 어느 지점에서 하나로 정리되나요?
3. 그 설계에서 보장하려던 범위와 포기한 부분은 무엇이었나요?

좋은 답변의 신호

- 키 저장 위치와 보관 기간, 본문 비교 기준을 구체적으로 설명한다
- 동시 요청을 막는 지점(유일 제약·잠금)과 재시도 조건을 구분해 설명한다

우려 신호

- 멍등성을 '같은 요청은 한 번만 처리한다'는 일반론으로만 설명한다
- 키 충돌 시 응답 코드나 본문 비교 기준을 제시하지 못한다

근거

이력서 연결

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?tab=resume#claim-8a3e3b3f-8a65-4129-a93c-366c566a75fe>

R-05 · 멍등 재전송

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-05>

Q10 선택 실패 디브리핑 원인 분석 · 8분

R-04 실행 기록에서는 수량 누락·0·음수 같은 요청이 400이 아니라 201로 처리되었습니다. 주문 생성 전에 입력을 검사하는 절차를 어떤 순서와 기준으로 두셨나요?

의도 검증 항목과 검증 시점을 어떻게 배치했는지, 검증 실패가 역등성 기록과 재고에 남지 않는지 확인한다.

꼬리 질문

1. 수량과 상품 id, 헤더 중 무엇을 먼저 검사하나요?
2. 검증 실패로 끝난 요청의 키를 기록에 남기지 않으려면 어느 시점에 기록해야 하나요?
3. JSON이 아닌 본문은 어느 단계에서 걸러지나요?

좋은 답변의 신호

- 검증 항목과 순서를 구체적으로 나열한다.
- 검증 실패 요청이 상태를 바꾸지 않아야 하는 이유를 말한다.

우려 신호

- 검증을 재고 확인 뒤에 두는 흐름을 그대로 답한다.
- 실패한 요청의 키가 기록에 남는지 여부를 고려하지 않는다.

근거

R-04 · 입력 검증

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-04>

R-04 재생

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-04&run=c0856d5f-4471-4b54-b7a8-5220110523ca>

src/app.ts:43-69

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L43-L69>

src/app.ts:23-31

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L23-L31>

Q11 선택 설계 트레이드오프 설계와 변경 용이성 · 6분

코드에 명시적 `any`가 3곳 있다는 신호가 있습니다. 이 지점들에서 타입을 `any`로 둔 이유와, 대신 쓸 수 있었던 타입 정의를 어떻게 판단하셨나요?

의도 `any`를 선택한 근거와 타입 경계 설계를 어떻게 판단했는지 확인한다.

꼬리 질문

1. 이 위치에서 타입을 좁히려면 어떤 정보가 더 필요했나요?
2. `any`를 남겨 둘 때 이후 변경에서 어떤 위험이 생기나요?
3. 요청 본문 타입을 어디에 정의하면 검증과 타입이 함께 맞춰지나요?

좋은 답변의 신호

- `any`를 둔 지점과 이유를 구체적으로 설명한다.
- 타입 정의 위치와 검증 규칙의 관계를 말한다.

우려 신호

- 타입 오류를 피하려고 `any`를 썼다고만 답한다.
- 해당 위치가 어디인지 짚지 못한다.

근거

src/app.ts:44-44

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L44-L44>

src/app.ts:90-90

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L90-L90>

src/data.ts:42-42

<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/data.ts#L42-L42>

R-12 · 설계·변경 용이성

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-12>

Q12 선택 요구사항 확장 트레이드오프 판단과 확장 · 6분 기본 질문

프로세스를 다시 시작해도 주문과 상태가 남아야 하는 상황을 가정해 보겠습니다. 지금 구현에서 가장 먼저 바뀌어야 할 부분은 어디인가요?

의도 요구사항이 바뀌었을 때 현재 구현(R-01·R-02·R-05)에서 영향을 받는 부분과 필요한 보장을 판단하는지 확인한다.

꼬리 질문

1. 지금 구현에서 그 상황의 영향을 받는 상태는 무엇인가요?
2. 그 상태를 안전하게 다루려면 어떤 보장이 필요한가요?
3. 그 보장을 얻는 방법들 가운데 어떤 기준으로 하나를 고르시겠어요?

좋은 답변의 신호

- 영향을 받는 상태와 코드 위치를 구체적으로 짚는다.
- 선택지마다 비용과 보장 수준을 비교해 하나를 고른다.

우려 신호

- 특정 기술 이름만 말하고 필요한 보장을 설명하지 않는다.
- 현재 구현이 받는 영향을 구체적으로 짚지 않는다.

근거

R-01 · 정상 주문 생성

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-01>

R-02 · 상품·주문 조회

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-02>

R-05 · 먹등 재전송

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-05>

Q13 선택 이력서 연결 트레이드오프 판단과 확장 · 5분 기본 질문

이 경험에서 본인이 맡은 범위와 가장 어려웠던 기술적 결정을 설명해 주시겠어요?

이력서: Node.js, Express를 이용한 REST API 설계와 구현 학습

의도 이력서에 적힌 경험에서 지원자가 직접 맡은 범위와 기술적 판단의 근거를 확인한다.

꼬리 질문

1. 그 작업에서 본인이 직접 설계하거나 구현한 부분은 어디까지였나요?
2. 그 결정을 내릴 때 어떤 기준으로 대안을 비교하셨나요?
3. 지금 다시 한다면 어떤 부분을 다르게 선택하시겠어요?

좋은 답변의 신호

- 본인이 맡은 범위와 팀이 맡은 범위를 구분해 설명한다
- 결정의 근거로 당시의 제약 조건과 비교한 대안을 구체적으로 든다

우려 신호

- 맡은 범위를 물어도 팀 전체의 결과만 설명한다
- 결정의 이유를 물으면 기술 이름만 나열하고 적용 조건을 말하지 않는다

근거

이력서 연결

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?tab=resume#claim-7dc08579-9209-48dc-b9dd-0fa193cdf155>

R-04 · 입력 검증

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-04>

북마크 API의 테스트는 어떤 입력과 기대 결과를 기준으로 작성하셨나요?

이력서: 인메모리 저장소와 간단한 테스트 코드를 포함합니다

의도 이력서에 적힌 테스트 코드의 검증 범위와 케이스 선정 기준을 확인한다.

꼬리 질문

1. 정상 경로 외에 어떤 실패 케이스를 테스트에 넣었나요?
2. 테스트가 통과해도 실제 결함이 남을 수 있는 부분은 어디라고 보시나요?
3. 인메모리 저장소를 쓰는 테스트에서 상태 초기화는 어떻게 처리했나요?

좋은 답변의 신호

- 경계값과 실패 케이스를 구체적으로 열거한다
- 테스트가 검증하지 못하는 범위를 스스로 구분해 설명한다

우려 신호

- 정상 경로 위주의 테스트만 언급한다
- 테스트 통과를 구현 정확성의 근거로만 설명한다

근거**이력서 연결**

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?tab=resume#claim-108ba4f5-ada1-4013-acd9-5b7622a94d3a>

G1 · 테스트 실효성: 경계·검증

<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=G1>

평가 척도 (역량별 4단계)

면접관이 기입할 때 쓰는 기준이며 시스템은 값을 채우지 않습니다.

경계 조건과 실패 처리

잘못된 입력과 자원 부족 같은 실패 경로를 예측하고 일관되게 처리한다.

- | | |
|----------------|---|
| 1 미흡 | 잘못된 입력이나 자원 부족 같은 실패 경로를 떠올리지 못한다. |
| 2 보완 필요 | 실패 경로를 묻으면 떠올리지만, 처리 방식과 응답 형식을 구체적으로 말하지 못한다. |
| 3 충족 | 경계값과 실패 경로를 스스로 나열하고, 각각의 응답 코드와 상태 변화를 설명한다. |
| 4 탁월 | 재시도 중 실패나 부분 실패처럼 실패가 겹치는 상황까지 다루고, 상태가 일관되게 남는 근거를 설명한다. |

동시성과 데이터 정합성

동시에 들어오는 요청에서도 데이터가 어긋나지 않도록 보호한다.

- | | |
|----------------|---|
| 1 미흡 | 동시에 들어온 요청 때문에 상태가 어긋날 수 있다는 점을 인식하지 못한다. |
| 2 보완 필요 | 경쟁 조건이 생길 수 있다는 점은 알지만, 어느 구간에서 생기는지 짚지 못한다. |
| 3 충족 | 경쟁이 생기는 구간과 그 구간을 보호하는 방법(락, 원자적 연산, 유일성 제약)을 코드 위치와 함께 설명한다. |
| 4 탁월 | 저장소가 바뀌거나 인스턴스가 늘어날 때 보호 방법이 깨지는 조건을 짚고, 대안을 비교해 설명한다. |

테스트 설계

결함을 실제로 잡아내는 테스트를 설계하고 빈틈을 스스로 찾는다.

- 1 미흡 작성한 테스트가 어떤 결함을 잡는지 설명하지 못한다.
- 2 보완 필요 정상 경로 테스트는 설명하지만, 살아남은 변이가 드러낸 빈틈을 스스로 찾지 못한다.
- 3 충족 살아남은 변이나 약한 단언을 보고, 추가할 테스트를 구체적인 입력과 기대값으로 말한다.
- 4 탁월 테스트의 우선순위를 정하는 기준을 설명하고, 테스트 비용과 결함을 잡는 범위의 균형을 논한다.

설계와 변경 용이성

책임을 나누어 요구가 바뀌었을 때 고칠 곳이 좁고 분명한 구조를 만든다.

- 1 미흡 코드 구조를 그렇게 나눈 이유를 설명하지 못한다.
- 2 보완 필요 구조는 설명하지만, 요구가 바뀌면 어디를 고쳐야 하는지 분명히 말하지 못한다.
- 3 충족 책임을 나눈 기준을 설명하고, 요구 변경 하나에 대해 고칠 위치를 코드에서 짚는다.
- 4 탁월 현재 구조의 한계를 스스로 짚고, 다른 구조와 비교해 선택에 따르는 비용을 설명한다.

원인 분석 면접에서만 확인

관측된 실패에서 근거를 따라 원인 코드까지 범위를 좁힌다.

- 1 미흡 재생 기록을 보고도 실패 원인을 추정하지 못한다.
- 2 보완 필요 원인 후보를 말하지만, 근거를 대지 못하고 추측에 머문다.
- 3 충족 재생 기록의 기대값·실제값에서 원인 코드 위치까지 스스로 좁힌다.
- 4 탁월 원인을 좁힌 뒤, 같은 원인으로 생길 수 있는 다른 결함과 재발을 막는 방법까지 제시한다.

트레이드오프 판단과 확장 면접에서만 확인

선택지의 비용과 이점을 기준을 세워 비교하고 조건이 바뀌면 판단을 조정한다.

- 1 미흡 선택지 사이의 차이를 설명하지 못하고 한 가지 방법만 제시한다.
- 2 보완 필요 선택지를 나열하지만, 무엇을 기준으로 골라야 하는지 말하지 못한다.
- 3 충족 정합성, 성능, 복잡도 같은 판단 기준을 밝히고, 과제 조건에서 그 방법을 고른 이유를 설명한다.
- 4 탁월 트래픽 증가나 다중 인스턴스처럼 조건이 바뀌면 선택이 어떻게 달라지는지 근거와 함께 설명한다.